R 语言基础（四）

条件语句、循环与自定义函数

郑泽靖

北京师范大学统计学院

2026 年秋季学期

上讲回顾：判断与随机抽样

比较运算得到逻辑值；`sample()` 可以模拟随机抽样。

输入代码

```
x <- 1
x < 2
sample(1:6, 1)
```

- `x < 2` 的结果是 `TRUE`。
- 抽样结果是 1 到 6 中的一个整数，每次可能不同。
- 本讲用条件控制执行，用循环完成重复计算。

条件语句的三种形式

条件语句根据判断结果决定执行哪段代码。

形式	执行规则
`if`	条件成立时执行
`if / else`	两种情形中执行一种
`if / else if / else`	依次判断，执行相应分支

`if` 的条件应得到一个 `TRUE` 或 `FALSE`。

单条件语句：if

条件不成立时跳过花括号中的代码；成立时执行。

输入代码

```
x <- 1
if (x < 1) {
  x <- x + 1
}
x
if (x < 2) {
  x <- x + 1
}
x
```

Console 输出

```
[1] 1
[1] 2
```

第一处不执行。
第二处执行。

两种情形：if / else

条件成立时执行前一分支，否则执行后一分支。

输入代码

```
x <- 1
if (x < 1) {
  x <- x - 1
} else {
  x <- x + 1
}
x
```

Console 输出

```
[1] 2
```

`else` 与前一分支的右花括号写在同一行。

多种情形：依次判断

遇到第一个成立的条件后，执行相应分支并结束本结构。

输入代码

```
x <- 1
if (x <= 0) {
  y <- 0
} else if (x <= 1) {
  y <- x
} else {
  y <- 2
}
y
```

Console 输出

```
[1] 1
```

执行第二分支。

教材例 2.A.3：计算分布函数

对给定的一个数值 x，计算

$$F(x)=\begin{cases}0, & x\le 0,\\ x, & 0<x\le 1,\\ 1, & x>1.\end{cases}$$

- 先判断 `x <= 0`。
- 该条件不成立后，再判断 `x <= 1`。
- 前两个条件都不成立时，取值为 1。

分布函数：输入与结果

输入代码

```
x <- 0.5
if (x <= 0) {
  Fx <- 0
} else if (x <= 1) {
  Fx <- x
} else {
  Fx <- 1
}
Fx
```

Console 输出

```
[1] 0.5
```

`Fx` 保存
本次计算结果。

分布函数：检查区间边界

修改 x，重新运行上一页的完整判断代码。

输入 x	进入的分支	结果 $F(x)$
-0.5	第一分支	0
0	第一分支	0
0.5	第二分支	0.5
1	第二分支	1
2	第三分支	1

for 循环：依次取值

for 循环让循环变量依次取向量中的值，
每取一个值就执行一次循环体。

1. 取向量中的第一个值。
2. 执行花括号中的程序代码。
3. 取下一个值，继续执行，直到向量中的值取完。

for 循环：输出 1、2、3

输入代码

```
for (i in 1:3) {
  print(i)
}
```

Console 输出

```
[1] 1
[1] 2
[1] 3
```

- i 是循环变量，依次取 1、2、3。
- print(i) 每次显示 i 的当前值。

for 循环：计算 1 到 5 的和

输入代码

```
s <- 0
for (i in 1:5) {
  s <- s + i
}
s
```

Console 输出

```
[1] 15
```

- 循环开始前，令 `s <- 0`。
- 每次将当前的 `i` 加入已有结果。

求和过程：每次更新结果

s 的初值为 0。

本次 i	本次计算	更新后的 s
1	$0+1$	1
2	$1+2$	3
3	$3+3$	6
4	$6+4$	10
5	$10+5$	15

教材例 2.A.5：重复计算骰子频率

先模拟 1000 次掷骰，再分别取前 n 次计算六种点数的频率。

输入代码

```
x <- sample(1:6, 1000, TRUE)
f <- c()
```

循环编号 i	使用的次数 n
$1, 2, \ldots, 10$	$10, 20, \ldots, 100$
11	500
12	1000

骰子频率：循环完成重复步骤

输入代码

```
for (i in 1:12) {
  if (i < 11) { n <- i * 10
  } else if (i == 11) { n <- 500
  } else { n <- 1000 }
  y <- x[1:n]
  f <- c(f, c(sum(y == 1), sum(y == 2),
              sum(y == 3), sum(y == 4),
              sum(y == 5), sum(y == 6)) / n)
}
```

每轮确定 n，提取前 n 次，再保存六种频率。

骰子频率：整理为矩阵

输入代码

```
f <- matrix(f, nrow = 12, ncol = 6,
            byrow = TRUE)
f[1:3, ]
```

- 每行对应一个 n，每列对应一种点数。
- byrow = TRUE 按行放入每轮的六个频率。
- 最后一行代码显示前三行，
  数值每次可能不同。

课堂练习一：条件与循环

练习时间：11 分钟。新建脚本 `R4-practice.R`。

1. 令 `a <- 2`。若 `a > 0`，将 `a` 加 1；显示结果。
2. 令 `x <- -2`。用 `if / else`：
   正数时令 `y <- 1`，否则令 `y <- 0`。
3. 用教材例 2.A.3 的三分支代码，计算 $x = 0.75$ 时的 $F(x)$。
4. 用 `for` 循环计算 $1 + 2 + \cdots + 10$。

保存脚本，核对计算结果。

练习一参考：三种条件语句

输入代码

```
a <- 2
if (a > 0) { a <- a + 1 }
x <- -2
if (x > 0) { y <- 1 } else { y <- 0 }
x <- 0.75
if (x <= 0) { Fx <- 0
} else if (x <= 1) { Fx <- x
} else { Fx <- 1 }
c(a, y, Fx)
```

Console 输出

```
[1] 3.00 0.00 0.75
```

练习一参考：循环求和

输入代码

```
s <- 0
for (i in 1:10) {
  s <- s + i
}
s
```

Console 输出

```
[1] 55
```

- 重新运行前，将 s 的初值重设为 0。
- 检查循环范围，并保存脚本。

第二节：按条件重复计算

打开上一节保存的 `R4-practice.R`，
在后面继续编写代码。

结构	控制重复的方式
`for`	循环变量依次取向量中的值
`while`	每轮先判断条件，成立时继续执行

while 循环：先判断，再执行

条件成立时执行循环体，然后重新判断；不成立时结束。

输入代码

```
k <- 1
s <- 0
while (k <= 3) {
  s <- s + k
  k <- k + 1
}
s
```

Console 输出

```
[1] 6
```

while 循环：条件如何变化

上一页开始时，k = 1，s = 0。

判断时 k	$k \le 3$	新 s	新 k
1	TRUE	1	2
2	TRUE	3	3
3	TRUE	6	4
4	FALSE	结束循环	

k <- k + 1 使条件发生变化，最终结束循环。

初次判断不成立时，循环体一次也不执行。

教材例 2.A.4：掷骰直到出现 6

记录出现第一个 6 时，已经掷了多少次。

输入代码

```
n <- 0
y <- 0
while (y != 6) {
  y <- sample(1:6, 1)
  n <- n + 1
}
n
```

y 保存本次点数，n 累计次数；结果每次可能不同。

掷骰实验：取得 10 次观测值

输入代码

```
Xn <- 1:10
for (i in 1:10) {
  n <- 0
  y <- 0
  while (y != 6) {
    y <- sample(1:6, 1)
    n <- n + 1
  }
  Xn[i] <- n
}
Xn
```

掷骰实验：区分两层重复

- 内层 `while`：完成一次“掷到 6 为止”的实验。
- 外层 `for`：把整个实验重复 10 次。
- `Xn[i]`：保存第 `i` 次实验的次数。

`Xn` 中共有 10 个观测值，每个值都是一个正整数。

自定义函数：保存一段计算

把经常使用的代码写成函数，之后通过函数名调用。

输入代码

```
mysum <- function(n) {
  s <- n * (n + 1) / 2
  return(s)
}
```

- `mysum`：自定义的函数名。
- `n`：输入参数，本例取正整数。
- `return(s)`：返回计算结果。

调用函数：计算前 50 个自然数之和

先运行上一页的函数定义，再调用 `mysum()`。

输入代码

```
mysum(50)
```

Console 输出

```
[1] 1275
```

$$1 + 2 + \cdots + 50 = \frac{50 \times 51}{2} = 1275.$$

更换参数，复用同一函数

输入代码

```
mysum(5)
mysum(10)
```

Console 输出

```
[1] 15
[1] 55
```

函数定义保持不变，更换括号中的输入值即可。

课堂练习二：while 与函数

练习时间：9 分钟。在 `R4-practice.R` 中继续完成。

1. 用 `while` 循环计算 $1+2+3+4$。设置初值，并在每轮更新循环变量。
2. 定义教材中的 `mysum(n)`，分别计算前 10 个、前 100 个自然数之和。
3. 核对结果，保存脚本。

练习二参考：while 求和

输入代码

```
k <- 1
s <- 0
while (k <= 4) {
  s <- s + k
  k <- k + 1
}
s
```

Console 输出

```
[1] 10
```

练习二参考：定义并调用函数

输入代码

```
mysum <- function(n) {
  s <- n * (n + 1) / 2
  return(s)
}
mysum(10)
mysum(100)
```

Console 输出

```
[1] 55
[1] 5050
```

本讲内容核对

结构	本讲用途
`if`	根据条件执行相应分支
`for`	循环变量依次取向量中的值
`while`	条件成立时重复执行
`function`	定义函数，传入参数并返回结果

保存 `R4-practice.R`，保留本次练习代码。

概率计算参考

- 数值积分与均匀分布的期望。
- 二项分布、正态分布的相关函数。
- 概率、分位数与随机数的计算。

供后续概率课使用，不计入本讲两节 45 分钟。

数值积分：integrate()

计算 $\int_0^1 x^2 \, \mathrm{d}x$。

输入代码

```
result <- integrate(function(x) x^2,
                    lower = 0, upper = 1)
print(result)
```

Console 输出

```
0.3333333 with absolute error < 3.7e-15
```

前面的数值是积分的近似值，后面给出数值误差信息。

提取积分的计算值

`integrate()` 的结果含多个分量，可以用 `$value` 提取积分值。

输入代码

```
result <- integrate(function(x) x^2,
                    lower = 0, upper = 1)
result$value
```

Console 输出

```
[1] 0.3333333
```

均匀分布的期望：教材例 2.3.9

若 $\xi \sim U(0, 60)$，理论期望为 $\mathrm{E}(\xi) = 30$。

输入代码

```
result <- integrate(function(x)
  x * dunif(x, min = 0, max = 60),
  lower = -Inf, upper = Inf)
result$value
```

Console 输出

```
[1] 29.99986
```

`dunif()` 给出密度值；数值积分得到近似结果。

概率分布函数的名称

常用分布的相关函数通常以 d、p、q、r 开头。

前缀	用途
d	离散分布的概率值，或连续分布的密度值
p	分布函数值，即 $P(X \leq x)$
q	给定累计概率，计算相应分位数
r	生成服从该分布的随机数

例如：binom 表示二项分布，norm 表示正态分布。

二项分布：概率与累计概率

设 $X \sim B(10, 0.25)$，
计算恰好答对两题、至多答对两题的概率。

输入代码

```
dbinom(2, size = 10, prob = 0.25)
pbinom(2, size = 10, prob = 0.25)
```

Console 输出

```
[1] 0.2815676
[1] 0.5255928
```

两个结果分别为 $P(X = 2)$ 与 $P(X \leq 2)$。

二项分布：生成随机数

用 `rbinom()` 生成服从二项分布的随机数。

输入代码

```
rbinom(2, size = 10, prob = 0.25)
```

- 第一个参数 2 表示生成两个随机数。
- 每个随机数表示 10 次试验中的成功次数。
- 成功概率为 0.25，结果每次可能不同。

标准正态分布：密度与概率

设 $Z \sim N(0,1)$，`sd = 1` 表示标准差为 1。

输入代码

```
dnorm(0.5, mean = 0, sd = 1)
pnorm(1.96, mean = 0, sd = 1)
```

Console 输出

```
[1] 0.3520653
[1] 0.9750021
```

第一项是 0.5 处的密度值；第二项是 $P(Z \leq 1.96)$。

标准正态分布：分位数与随机数

给定累计概率 0.975，计算相应分位数。

输入代码

```
qnorm(0.975, mean = 0, sd = 1)
```

Console 输出

```
[1] 1.959964
```

生成三个标准正态随机数，结果每次可能不同。

输入代码

```
rnorm(3, mean = 0, sd = 1)
```